

AI 활용 기술문서 (초안)

NAN2026 사전과제 제출용 5종 중 하나 — 제출 마감 2026-08-10 작성: 배영환·송원호 (Claude Code 세션 기록 기반) · 최종 갱신: 2026-08-05 대상 게임: **돌려돌려 그림자** 상태: **Draft** — 스크린샷 첨부, 라이선스 확인, PDF 변환 필요 (§8 TODO 참고)

1. 개요

- 사용 AI 도구:** Claude Code (CLI 에이전트), MCP 확장 도구(`chrome-devtools` — 실제 브라우저 자동 조작·스크린샷·콘솔/네트워크 로그 확인)
- 활용 범위:** 설계 결정(ADR 작성·대안 비교) · 구현(TypeScript/React/Canvas) · 버그 발견 및 수정 · 검증 (단위 테스트 + 브라우저 실측) · 문서화 · 커밋/PR 워크플로우 자동화
- 정량 지표** (2026-08-03 기준): 총 커밋 82개, 머지된 PR 12건, ADR 4건, 테스트 파일 11개(단위 테스트 107개)

2. 프로젝트 구조와 AI 워크플로우

레포 안에 Claude Code 전용 하네스(`CLAUDE.md`, `docs/ref/`, `.claude/`, `skills/`)를 구성해, 매 세션마다 같은 규칙(코드/커밋 언어, 보안, 모델 라우팅, MVP 범위 제한)을 강제했다.

- Plan 모드:** 코드 변경이 필요한 작업은 먼저 Plan 모드로 들어가 탐색(Explore) → 설계(Plan 에이전트) → 사용자 승인(ExitPlanMode) 후에만 실행 — 큰 구조 변경(라운드-스테이지 분리, 이동 패턴 AI 제거 등)에 이 절차를 적용해 계획 없이 코드부터 손대는 것을 방지했다.
- /commit, /PR 스킴:** 커밋 메시지·PR 본문을 컨벤션(`docs/ref/commit-convention.md`)에 맞춰 자동 생성 — `Rejected: / Confidence:` 같은 trailer로 "왜 이 방식을 안 썼는지"까지 커밋 이력에 남겼다.
- MVP 범위 제한**(`CLAUDE.md`)을 프로젝트 지침에 명시해, 회원가입·랭킹·마우스 조작 등 범위 밖 기능을 AI가 임의로 제안/구현하지 않도록 가드레일을 걸었다.

3. 주요 설계 결정 사례 (ADR)

AI가 코드를 먼저 쓰지 않고, 대안을 표로 비교한 뒤 결정 근거를 ADR로 남기는 방식을 반복 사용했다.

ADR	결정	비고
ADR-001	그림자 경계 판정 방식	초기 컨셉 확정 단계
ADR-002	캐릭터·그림자 이중 조작(각도 허용치 모델)	단일 판정 모델에서 재설계
ADR-003	타일 그리드 + Carve + 판정자 주입	5개 대안 비교 표, 이후 재검토 조건 2건이 실제로 트리거되어 라운드-스테이지 분리로 이어짐
ADR-004	다중 광원(가로등) — 최근접 광원 기준 안전구역	광원 배치 겹침·체크포인트 판정 순서 등 옛지 케이스 3건을 추가 기록으로 반영. 단, 광원 전환 순간 유예(<code>LIGHT_SWITCH_GRACE_SECONDS</code>)는 같은 커밋(<code>105923b</code>)에서 함께 구현됐지만 ADR 본문엔 반영되지 않음 — ADR엔 "활성 광원 강조 표시"만 완화책으로 적혀 있어, 코드와 문서가 어긋난 상태(별도 정리 필요)

4. AI 주도 버그 발견·수정 사례

브라우저 자동화(스크린샷·픽셀 캡처·시드 sweep)로 재현 → 근본 원인 분석 → 수정 → 회귀 테스트 추가의 흐름을 반복했다.

- 그림자 렌더링 퇴화 버그** — `shadowAngle` 이 정확히 수평($0^\circ/180^\circ$)일 때 전단(shear) 변환의 x/y축 기저벡터가 겹쳐 그림자가 화면에서 완전히 사라짐. 캔버스 픽셀 캡처로 재현($0^\circ \rightarrow$ 비가시, $90^\circ \rightarrow$ 정상) 후 `shadowVerticalComponent()` 로 최소 수직 성분을 강제해 수정. 회귀 테스트 4건 추가.
- 진척도 판정이 벽을 무시하던 버그** — 체크포인트/굴 통과 판정이 유클리드 최근접 방식이라, 벽 하나 너머의 먼 지점을 "가장 가깝다"고 오판(300시드 전수 실측으로 확인, 프로덕션 기본 시드도 포함). 스폰 기준 BFS 그래프 거리 필드로 교체해 벽을 넘는 오판을 구조적으로 차단. 200시드 sweep 테스트로 검증.
- 체크포인트 좌표 겹침 / 라운드 시드 충돌** — 넓은 시드 sweep 중 막다른 통로 재방문이 체크포인트 인덱스와 우연히 겹치는 경우, 재시도 시드 파생 함수가 라운드 파생 함수와 우연히 충돌해 $1R=2R$ 이 되는 경우를 각각 발견해 수정(`hasDistinctCells` , `deriveRoundSeed` 분리).
- 밝은 적 없는 세이브 포인트가 활성화되던 버그** — 플레이 중 "찍지도 않은 세이브 포인트가 활성화 표시되고 죽으면 처음 보는 지점에서 재개된다"는 제보에서 출발. 활성화 판정이 방향 정보가 없는 스칼라(스폰 기준 BFS 거리) 비교라 체크포인트와 정반대 갈래로 가도 통과 처리되는 것이 원인이었다. **체크포인트 주변을 통행 금지로 막고 BFS를 다시 돌리는** 방식으로 "근처에 얼씬도 않고 통과 가능한가"를 정량화해 전 라운드 6/6 재현을 확인했고, 판정을 실제 접촉(반경 40px)으로 교체해 원리적으로 차단했다. 같은 분석에서 배치 척도(carve 순서)와 판정 척도(BFS 거리)가 서로 달라 체크포인트가 코스 초반에 몰려 있던 2차 문제도 함께 드러나, 배치를 "스폰→굴 최단 경로 위 + 진행도 균등 분할"로 통일했다. 이때 **"밝기까지 더 걸어야 하는 거리"를 지표로 정량화**한 것이 결정적이었다 — 1차 수정(진행도 기준만 적용)은 50% 지점을 정확히 맞췄지만 체크포인트가 최단 경로 밖 막다른 갈래에 놓여 실질적으로 도달 불

가였고, 이 지표로 재보고 나서야 후보를 최단 경로 위로 제한하는 2차 수정이 나왔다(추가 이동 중앙값 -5px·최대 30px, 우회 가능성은 100/100 시드 유지).

상세 재현 조건·수정 내역은 각 ADR의 "버그 수정 기록" 절 참고.

5. 검증 방법론 — 헤드리스 시뮬레이션 + 실제 브라우저 이중 검증

1R→2R→3R 라운드 전환이 실제로 동작하는지 확인할 때, 미로를 정밀하게 자동 조작하기 어렵다는 문제에 두 단계로 대응했다.

1. 헤드리스 시뮬레이션: GameCanvas 가 쓰는 실제 프로덕션 함수

(`generateStage` / `moveCharacter` / `naturalAngle` / `alignmentMargin` / `roundAfterClear` 등)를 React/Canvas 없이 그대로 조합한 "가상 조종사" 봇을 임시 테스트로 작성 — 로직 자체가 올바른지 빠르게 검증(검증 후 파일 삭제, 커밋에 남기지 않음).

2. 실제 브라우저 검증: 헤드리스 결과만으로는 렌더링·입력 파이프라인까지는 보장되지 않는다는 지적을 받아, 임시 디버그 혹은 코드에 추가하고 `chrome-devtools` MCP로 실제 `KeyboardEvent` 를 dispatch해 진짜 게임을 실시간 자동 플레이 — 1R(19.4초)→2R(22.1초)→3R(12.0초)→엔딩까지 스크린샷으로 확인 후, 디버그 혹은 diff 없이 완전히 원복.

이 과정에서 부수적으로 3R(허용각도 0.5배 디메리트)에서 광원에 최소거리(~40px)로 붙어 지나갈 때 요구되는 회전 속도가 실제 회전속도(180도/초)에 근접/초과할 수 있어, 멈춰서 재정렬하지 않으면 통과가 매우 어려운 구간이 존재한다는 밸런스 인사이트를 발견했다 — 버그는 아니지만 P1 플레이테스트 항목으로 기록해두었다.

6. AI 활용의 한계와 사람의 검증

- **UX 판단은 사람이 결정:** 세이프 포인트 통과 피드백을 "대사창(즉시 확인 가능하지만 이동을 멈춤)" vs "색상 변화(비침투적)" 중 고르는 문제에서, 실시간 반사신경 게임이라는 장르 특성을 근거로 색상 변화만 채택 — AI가 두 옵션을 모두 제시하되 최종 선택은 사람이 했다.
- **임시 코드는 흔적 없이 원복:** 검증 목적으로 추가한 디버그 혹은 헤드리스 테스트 파일은 실제 커밋에 포함되지 않도록 `git diff` 로 매번 확인 후 제거.
- **모르는 건 추측하지 않음:** PRD 미결 사항(그림자 길이·통로 폭 등)은 AI가 임의로 "적당한 값"을 확정하지 않고, 플레이테스트로 팀이 결정해야 할 항목으로 명시적으로 남겨둬.

7. 콘텐츠/QA — 외부 AI 이미지 생성 활용 (원호)

작성: 송원호 (Claude Code 세션 기록 기반)

7.1 개요

- **사용 AI 도구:** GPT(DALL-E, 이미지 생성) — 캐릭터 디자인 시트·배경·바닥/벽 타일 텍스처 전량. Claude Code는 프롬프트 설계·이미지 검증(seamless 확인, 스케일 계산)·코드 통합. `chrome-devtools` / Puppeteer MCP를 통한 실제 브라우저 렌더링 검증을 담당.
- **원본 생성 4건 → 최종 반영 파일 18개:** 캐릭터 디자인 시트 1장(→ 파생 스프라이트 15장), 배경 1장 (`stage.png`, 라운드 전환은 밝기 오버레이로 재사용해 별도 생성 안 함), 타일 2장(floor/wall).
- **출처·라이선스는 전량 `public/assets/ASSET_SOURCES.md` 에 기록** — 파일별 생성 도구·날짜·약관 상태를 한 곳에서 추적.

7.2 캐릭터 스프라이트 파이프라인

GPT로 턴어라운드·표정 6종·동작 5종·죽음 모션 4단계를 한 장의 디자인 시트로 생성한 뒤, `scripts/crop-regions.mjs` (1회성 개발 스크립트, `sharp` 사용)로 좌표를 지정해 실제 코드가 쓰는 15장을 잘라냈다. 시트 원본은 `public/` 이 아니라 `scripts/` 에 뒤서(PR #9 리뷰 반영) 런타임에 로드되지 않는 원본이 배포 용량에 포함되지 않도록 했다.

- **걸기 애니메이션 레이어 분리:** 다리 단독 회전 → 전신 좌우 반전 → 2프레임 이미지 교체 순으로 시도했으나 전부 "진짜 걷는 것처럼" 보이지 않아 폐기. 최종적으로 상체(코트+얼굴, 이동 방향에만 반전)와 다리(킥아웃, 프레임마다 좌우 반전)를 별도 레이어로 분리 렌더링하는 방식으로 정착 (`characterSprites.ts` / `GameCanvas.tsx` 주석에 폐기 경위 기록).
- **크롭 버그:** `crop-regions.mjs` 의 배경 장식(잉크 방울·바닥 그림자·속도선) 제거용 `erase` 사각형이 머리·발·다리 실루엣까지 함께 지워버리는 문제를 실제 스프라이트 렌더링 스크린샷으로 발견해 좌표를 재조정(`eb11eab`).

7.3 배경·타일 텍스처 — 프롬프트 가이드 문서화

배경(`background-art-prompt-guide.md`)과 타일(`tile-art-prompt-guide.md`) 모두 "AI에게 무엇을 요청할 지"를 먼저 문서로 못박고 시작했다 — 프롬프트를 매번 즉흥적으로 짜면 재생성할 때마다 결과가 흔들리기 때문에, 공통 스타일 앵커(팔레트·조명 원칙)와 기술 제약(해상도, seamless 여부, 방향 무관성)을 재사용 가능한 템플릿으로 고정했다.

- **바닥/벽을 "통짜 맵 이미지"가 아니라 타일셋으로 요청한 이유:** 스테이지가 시드 기반 절차적 생성이라 통로 모양이 매번 달라진다 — AI에게 전체 맵을 그려달라고 하면 실제 절차적 결과와 절대 일치하지 않는다. 대신 셀 하나 분량의 반복 가능한 텍스처만 받아 코드가 그리드 좌표에 맞춰 반복 배치하는 방식으로 설계(`tile-art-prompt-guide.md` §"왜 통짜 맵 이미지가 아니라 타일셋인가").
- **스케일 보정:** 처음엔 바닥에서 쓴 것과 같은 시각적 가늠 방식을 벽에도 그대로 적용했더니 벽이 얇은 줄무늬로 뭉개져 안 읽혔다 — 바닥(날장 판석 무늬가 그리드 1칸에 대략 하나씩 오도록 가늠, `FLOOR_TILE_SCALE=0.16`, 원본 1254px 기준 약 200px 주기)과 달리 벽은 무늬 반복 주기가 훨씬 커서(세로 방향 픽셀 밝기 프로파일을 직접 추출해 실측한 결과 $\approx 730\text{px}$) 같은 비율로는 안 맞았던 것. 벽만 별도로 실측치를 반영한 스케일 값(`WALL_TILE_SCALE`)을 산출.
- **배포 용량 최적화:** PR #20 리뷰(배영환)에서 원본 1254×1254px 이미지가 실제 화면에는 16~30%만 쓰이는데 그대로 커밋돼 용량이 과다하다는 지적을 받아, 512×512px로 리사이즈(약 1/7 용량 축소) — 리사이즈 비율만큼 스케일 값을 반비례 보정해 화면상 반복 주기는 그대로 유지했다.

7.4 검증 방법론 — 2×2 이음매 확인 + 실제 브라우저 렌더링

- **seamless 검증**: 후보 이미지를 2×2로 이어붙인 합성 이미지를 만들어 타일 경계에 눈에 띄는 이음매가 있는지 확인한 뒤에만 코드에 반영.
- **게임 화면 실측**: `chrome-devtools / Puppeteer MCP`로 실제 개발 서버를 조작해 — 캐릭터 이동, 안전구역 가이드라인(보라색 부채꼴), 세이브 포인트, 광원 웅덩이 등이 새 텍스처 위에서도 대비·가독성을 유지하는지 스크린샷으로 확인. 배경/타일 로더는 `backgroundArt.ts / tileArt.ts` 에서 로드 실패 시 `null` 을 반환하고 렌더러가 기존 절차적 렌더링(단색·procedural grain)으로 자동 폴백하도록 해, 이미지 로드 실패가 게임 플레이 자체를 막지 않도록 방어했다.

7.5 라이선스 관리와 남은 확인 사항

- 이미지를 추가할 때마다 `public/assets/ASSET_SOURCES.md` 에 파일 경로·생성 방법·라이선스 조건·추가일을 한 줄씩 기록하는 규칙을 세워 전량 추적 가능하게 유지.
- **제출 전 미확인 상태로 남은 것**(`tile-art-prompt-guide.md / background-art-prompt-guide.md` 체크리스트): (1) GPT 생성 이미지의 최신 소유권·상업적 이용 약관 확인, (2) NAN2026 대회 규정상 AI 생성 에셋 허용 여부·출처 명시 의무 확인 — 둘 다 외부 문서(OpenAI 약관, 대회 규정집) 확인이 필요해 AI가 대신 판단할 수 없는 항목으로, 팀이 제출 전 직접 확인해야 한다.

8. TODO (제출 전 보강 필요)

- ☒ 원호님 파트 추가 — 배경 아트 프롬프트 가이드(외부 AI 이미지 생성 활용) 등 콘텐츠/QA 쪽 AI 활용 사례
- ☐ 스크린샷 첨부 (헤드리스 검증 로그, 브라우저 자동 조종 스크린샷 등)
- ☐ GPT 생성 이미지 이용약관·해커톤 AI 에셋 규정 확인 (§7.5 참고)
- ☐ 최종 마크다운 → PDF 변환